Faculty of Business and Technology: Programme – Cyber Security and Digital Forensics

NAME: Zuriel Etim

STUDENT NUMBER:229105670

MODULE CODE: CET 300

MODULE TITLE: Computing Project

MODULE LEADER: Becky Allen

SUBMISSION DATE AND TIME: 09/05/2025

ASSIGNMENT: User Guide

Academic Misconduct is an offence under university regulations, and this involves:

- **Plagiarism** – where you use information from another information source (including your previously submitted work) and pass it off as your own. This can be through direct copying, poor paraphrasing and/or absence of citations.
- **Collusion** – where you work too closely, intentionally, or unintentionally, with others to produce work that is similar in nature. This can be through loaning of materials, drafts or through unauthorised use of a fellow student's work.
- **Asking another person to write your assignment** – where you ask another individual or company to complete your work for you, be that paid or unpaid, and submit it as if it were your own.
- **Unauthorised use of artificial intelligence** – where you use artificial intelligence tools to generate your assignment instead of completing it yourself and/or where you have not been given permission to use artificial intelligence tools by your module leader. Please complete the following declaration around you use of artificial intelligence tools in your assignment.

STATEMENT ON USE OF ARTIFICIAL INTELLIGENCE TOOLS:

- I have used artificial intelligence tools to generate an idea for my assignment: **NO**
- I have used artificial intelligence tools to write my assignment for me: **NO**
- I have used artificial intelligence tools to brainstorm ideas for my assignment: **NO**
- I have used artificial intelligence tools to correct my original assignment: **NO**

DECLARATION

- I understand that by submitting this piece of work I am declaring it to be my own work and in compliance with the university regulations on Academic Integrity.

- I confirm that I have done this work myself without external support or inappropriate use of resources.
- I understand that I am only permitted to use artificial intelligence tools in line with guidance provided by my Module Leader, and I have not used artificial intelligence tools outside this remit.
- I confirm that this piece of work has not been submitted for any other assignment at this or another institution prior to this point in time.
- I can confirm that all sources of information, including quotations, have been acknowledged by citing the source in the text, along with producing a full list of the sources used at the end of the assignment.
- I understand that academic misconduct is an offence and can result in formal disciplinary proceedings.
- I understand that by submitting this assignment, I declare myself fit to be able to complete the assignment and I accept the outcome of the assessment as valid and appropriate.

ANY OTHER COMMENTS FROM STUDENT:

Zuriel's Honey Pot Manual

1. Open the Virtual Machine and Log in. Username is Kali and Password is Kali

Cowrie

Firstly we need to enter the directory that contains the Cowrie Honeypot.

"cd cowrie" the path should be "/home/kali/cowrie"

Then cowrie would be able to be booted

"bin/cowrie start"

```
┌──(kali㉿kali)-[~/cowrie]
└─$ bin/cowrie start
Using default Python virtual environment "/home/kali/cowrie/cowrie-env"
Starting cowrie: [twistd  --umask=0022 --pidfile=var/run/cowrie.pid --logger cowrie.python.logfile.logger cowrie ]...
/home/kali/cowrie/cowrie-env/lib/python3.13/site-packages/twisted/conch/ssh/transport.py:105: CryptographyDeprecationWarning: TripleDES
n cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  b"3des-cbc": (algorithms.TripleDES, 24, modes.CBC),
/home/kali/cowrie/cowrie-env/lib/python3.13/site-packages/twisted/conch/ssh/transport.py:112: CryptographyDeprecationWarning: TripleDES
n cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  b"3des-ctr": (algorithms.TripleDES, 24, modes.CTR),
Another twistd server is running, PID 53558

This could either be a previously started instance of your application or a
different application entirely. To start a new one, either run it in some other
directory, or use the --pidfile and --logfile parameters to avoid clashes.
```

Splunk

- Firstly we need to start up the Log Visualiser, Splunk to set up the real time alert for both the Brute Force Logs from the Cowrie Honeypot

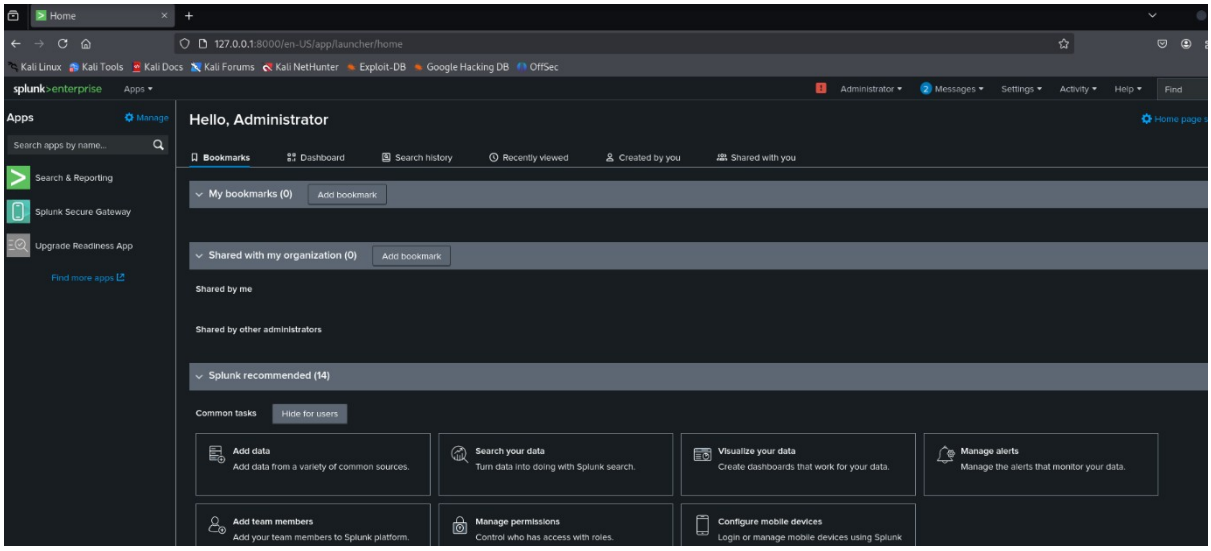
- Splunk has been set up so that it runs when the honeypot boots up, so open up the web browser and enter this URL to access Splunk

http://127.0.0.1:8000

- But if for some reason it hasn't started up run this command in the terminal and put in the URL above into the web browser.

Sudo /opt/splunk/bin/splunk start

Username: "admin"
Password: "admin123"



Towards the left of the screen you'll see Search and Reporting; click that enter this search term.

source="/home/kali/cowrie/var/log/cowrie/cowrie.log" host="kali" sourcetype="Cowrie Log"

This is a basic search filter that tells Splunk to look for logs from the Cowrie honeypot.

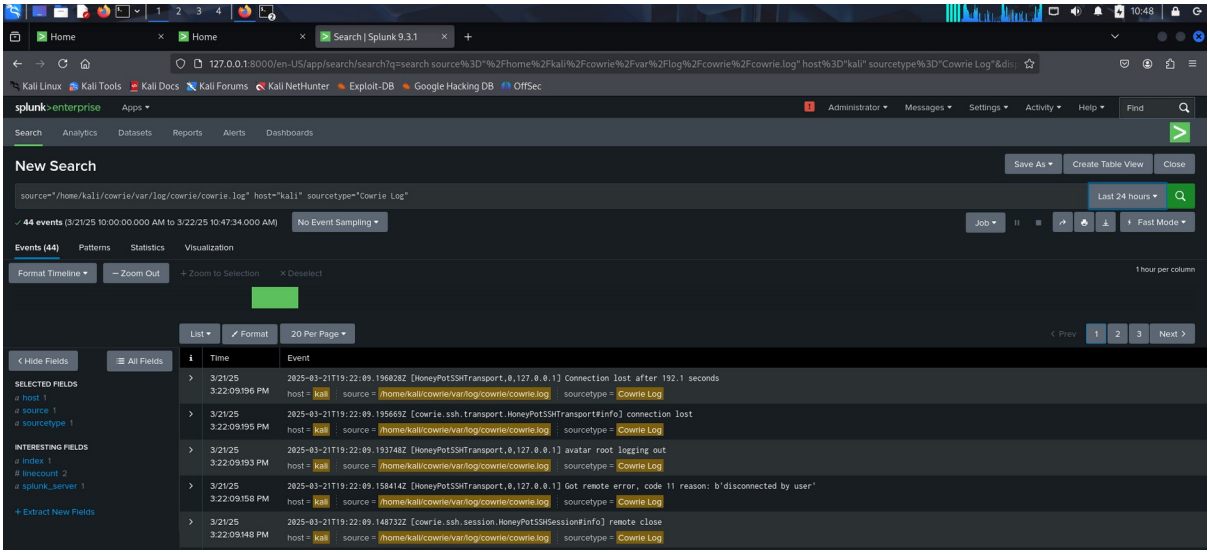
source="/home/kali/cowrie/var/log/cowrie/cowrie.log"

- This tells Splunk to only look at logs from **this specific file** on the machine.
- cowrie.log contains **general honeypot activity**, such as login attempts, file interactions, and attacker behavior.

host="kali"

- The **host field** is the name of the system sending the logs.
- In this case, it's "kali", meaning logs are coming from a machine named "kali" in Splunk.

The outcome is below, showing all the logs of the cowrie logs, displaying all the connections made, the IP addresses that made the connection and the actions the attacker made



The search then displays all the logs of the cowrie logs, displaying all the connections made, the IP addresses that made the connection and the actions the attacker made while connected to the honeypot with changes.

- When your satisfied with everything. Click ‘Save As’ and tap New Dashboard
  Fill out the details and click save to DashBoard

  Instead of manually running searches every time, a dashboard updates automatically to display the latest attack data.
  The dashboard can be set with refresh intervals (e.g., every 30 seconds) to keep an eye on brute-force attempts as they happen.

Brute Force Attack Test

On the attacker VM all the tools have been pre-installed already.

Firstly on the attacker VM, we would need to create a list of possible Usernames and Passwords to run them against the honeypot. This could be done with :

```
echo -e "root\admin\ntest" > usernames.txt
echo -e "root\n1234\admin\password" > passwords.
```

This should return two text files for the corresponding names

To conduct the attack, it would need to be carried out on the command line using Hydra. The command for this below;

```
hydra -L usernames.txt -P passwords.txt ssh:// 192.168.92.132 -s 2222
```

Expected Output:

C. Expected Result

Hydra will attempt combinations and Cowrie , will:

- Log all login attempts
- Record IP address of the attacker
- Log entered commands even if fake

```
┌──(kali㉿kali)-[~]
└─$ hydra -L usernames.txt -P passwords.txt ssh://192.168.92.132 -s 2222

Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
 military or secret service organizations, or for illegal purposes (this is n
on-binding, these ***  ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-04-11 06:
50:18
[WARNING] Many SSH configurations limit the number of parallel tasks, it is r
ecommended to reduce the tasks: use -t 4
[DATA] max 12 tasks per 1 server, overall 12 tasks, 12 login tries (l:3/p:4),
 ~1 try per task
[DATA] attacking ssh://192.168.92.132:2222/
[2222][ssh] host: 192.168.92.132   login: root   password: admin
[2222][ssh] host: 192.168.92.132   login: test   password: password
[2222][ssh] host: 192.168.92.132   login: admin   password: admin
[2222][ssh] host: 192.168.92.132   login: admin   password: password
[2222][ssh] host: 192.168.92.132   login: root   password: 1234
[2222][ssh] host: 192.168.92.132   login: admin   password: root
[2222][ssh] host: 192.168.92.132   login: test   password: root
[2222][ssh] host: 192.168.92.132   login: test   password: 1234
[2222][ssh] host: 192.168.92.132   login: root   password: password
[2222][ssh] host: 192.168.92.132   login: admin   password: 1234
[2222][ssh] host: 192.168.92.132   login: test   password: admin
1 of 1 target successfully completed, 11 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-04-11 06:
50:20
```

With the logs being able to be seen in the dashboard created or at the file location (/home/kali/cowrie/var/log/cowrie/cowrie.log)

Suricata

Open the command Line and enter the command below to start up Suricata

sudo systemctl start suricata

And to confirm its running properly put this command below

sudo systemctl status suricata

```
┌──(kali㉿kali)-[~]
└─$ sudo systemctl start suricata


┌──(kali㉿kali)-[~]
└─$ sudo systemctl status suricata

● suricata.service - Suricata IDS/IDP daemon
     Loaded: loaded (/usr/lib/systemd/system/suricata.service; disabled; preset: disabled)
     Active: active (running) since Fri 2025-05-09 17:30:33 EDT; 4s ago
 Invocation: effe455415454b2dae474008701e77b9
       Docs: man:suricata(8)
             man:suricatasc(8)
             https://suricata.io/documentation/
    Process: 8878 ExecStart=/usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricat
   Main PID: 8879 (Suricata-Main)
      Tasks: 1 (limit: 2199)
     Memory: 146M (peak: 146M)
        CPU: 4.987s
     CGroup: /system.slice/suricata.service
             └─8879 /usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid

May 09 17:30:33 kali systemd[1]: Starting suricata.service - Suricata IDS/IDP daemon...
May 09 17:30:33 kali suricata[8878]: i: suricata: This is Suricata version 7.0.10 RELEASE running in SYSTEM mode
May 09 17:30:33 kali systemd[1]: Started suricata.service - Suricata IDS/IDP daemon.
lines 1-18/18 (END)
```

Testing Suricata

The way were going to test Suricata is by generating basic malicious traffic to test the rule detection mechanisms.

Firstly we monitor the log files for any changes with this command

sudo tail -f /var/log/suricata/fast.log

Then open up another Terminal and input this curl command to generate the malicious traffic as we see below

```
┌──(kali㉿kali)-[~]
└─$
curl http://testmynids.org/uid/index.html
uid=0(root) gid=0(root) groups=0(root)

┌──(kali㉿kali)-[~]
└─$
curl http://testmynids.org/uid/index.html
uid=0(root) gid=0(root) groups=0(root)
```

The log file should look something like this

```
┌──(kali㉿kali)-[~]
└─$ sudo tail -f /var/log/suricata/fast.log
05/09/2025-17:41:05.485309  [**] [1:2100498:7] GPL ATTACK_RESPONSE id check returned root [**] [Classification: Pote
tially Bad Traffic] [Priority: 2] {TCP} 18.245.162.94:80 → 192.168.92.132:51128
05/09/2025-17:41:10.071650  [**] [1:2100498:7] GPL ATTACK_RESPONSE id check returned root [**] [Classification: Pote
tially Bad Traffic] [Priority: 2] {TCP} 18.245.162.94:80 → 192.168.92.132:37958
```

This shows that it is able to alert and log the traffic that was generated.